Dokumentation Windows OPC UA

Muhammed Hamzic, Semin Sabanovic, Mirza Dzinic

April 2026

Inhaltsverzeichnis

1 Installation von Zip **1**

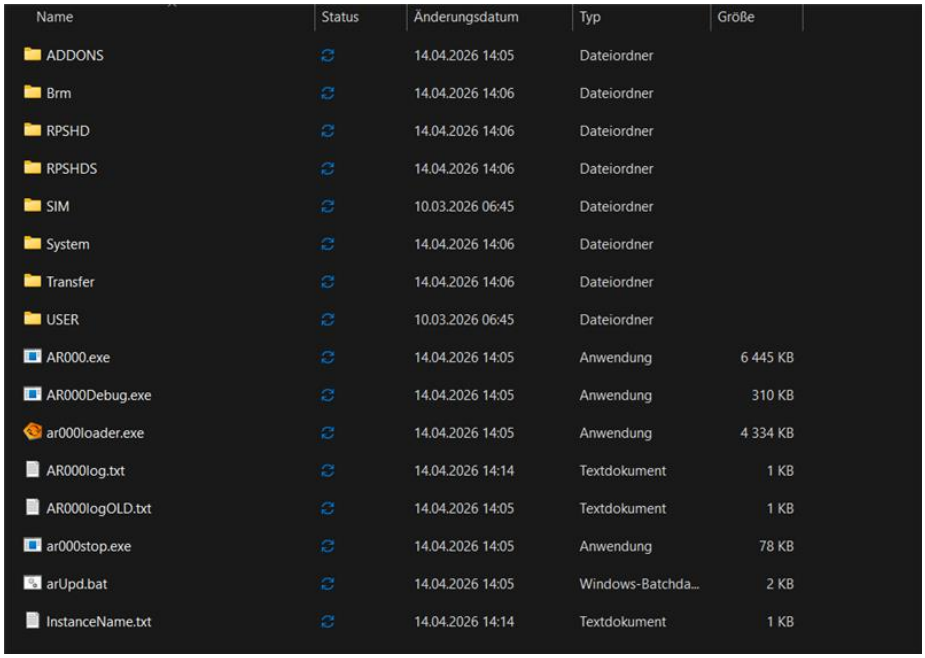
2 UaExpert-Steuerung **3**

3 Zusammenfassung **7**

Kapitel 1

Installation von Zip

Wir haben die .zip Datei von Teams heruntergeladen, welche wir vom Schweiberer zur Verfügung gestellt bekommen haben.



Name	Status	Änderungsdatum	Typ	Größe
ADDONS		14.04.2026 14:05	Dateiordner	
Brm		14.04.2026 14:06	Dateiordner	
RPSHD		14.04.2026 14:06	Dateiordner	
RPSHDS		14.04.2026 14:06	Dateiordner	
SIM		10.03.2026 06:45	Dateiordner	
System		14.04.2026 14:06	Dateiordner	
Transfer		14.04.2026 14:06	Dateiordner	
USER		10.03.2026 06:45	Dateiordner	
AR000.exe		14.04.2026 14:05	Anwendung	6 445 KB
AR000Debug.exe		14.04.2026 14:05	Anwendung	310 KB
ar000loader.exe		14.04.2026 14:05	Anwendung	4 334 KB
AR000log.txt		14.04.2026 14:14	Textdokument	1 KB
AR000logOLD.txt		14.04.2026 14:05	Textdokument	1 KB
ar000stop.exe		14.04.2026 14:05	Anwendung	78 KB
arUpd.bat		14.04.2026 14:05	Windows-Batchda...	2 KB
InstanceName.txt		14.04.2026 14:14	Textdokument	1 KB

Abbildung 1.1: Screenshot vom .zip Folder (Inhalt)

Nachdem man die Datei installiert und extrahiert hat, führt man die Datei **ar000loader.exe** aus. Nachdem die Datei ausgeführt wurde, öffnet sich ein kleines Fens-

ter. Dieses Fenster ist eine kleine Simulation von einer Windows-Steuerung, die Signale empfängt, diese weiterleitet und verarbeitet. Darin befinden sich Daten wie eine IP-Adresse, die man eigenständig ändern kann.

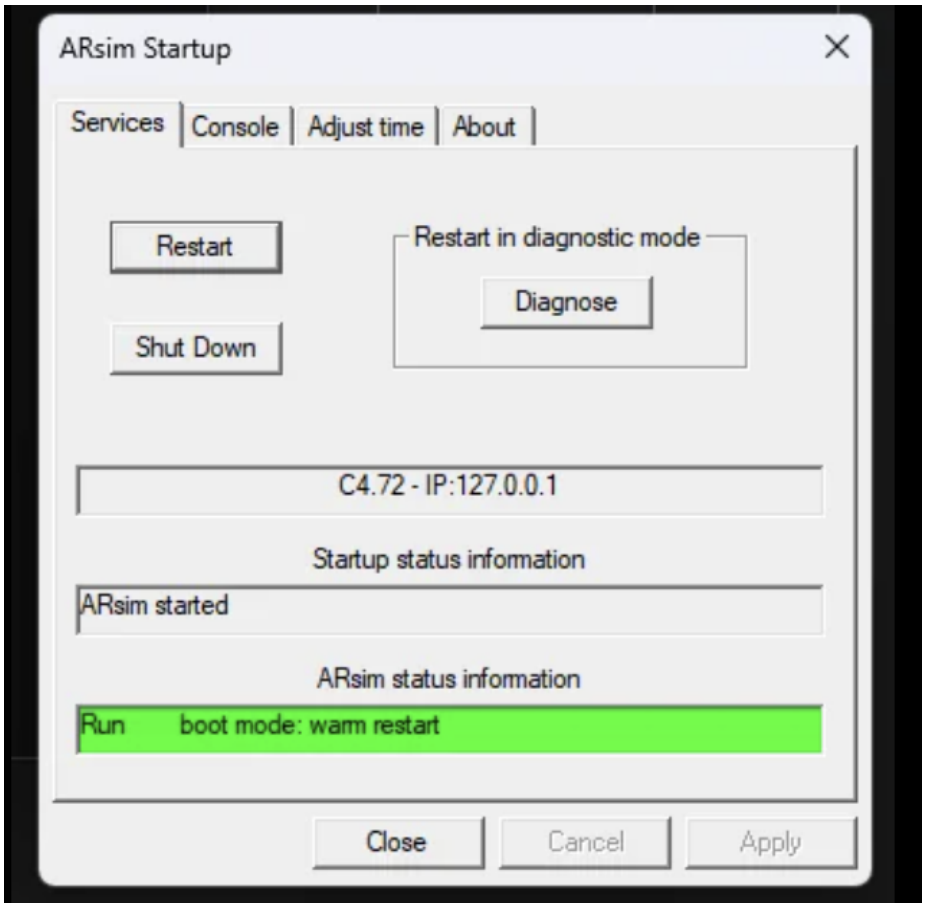


Abbildung 1.2: Screenshot vom Arsim Programm

Kapitel 2

UaExpert-Steuerung

In Unity öffnet man das Smarthome-Projekt und ändert im Script die IP-Adresse zu 127.0.0.1. Im UaExpert wird zunächst nichts angezeigt. In Unity sollte man das Script **“OpcUaClientBehaviour.cs“** in irgendeinem Editor seiner Wahl öffnen. Nachdem erfolgreichen Öffnen sollte der Code ohne Fehler angezeigt werden.

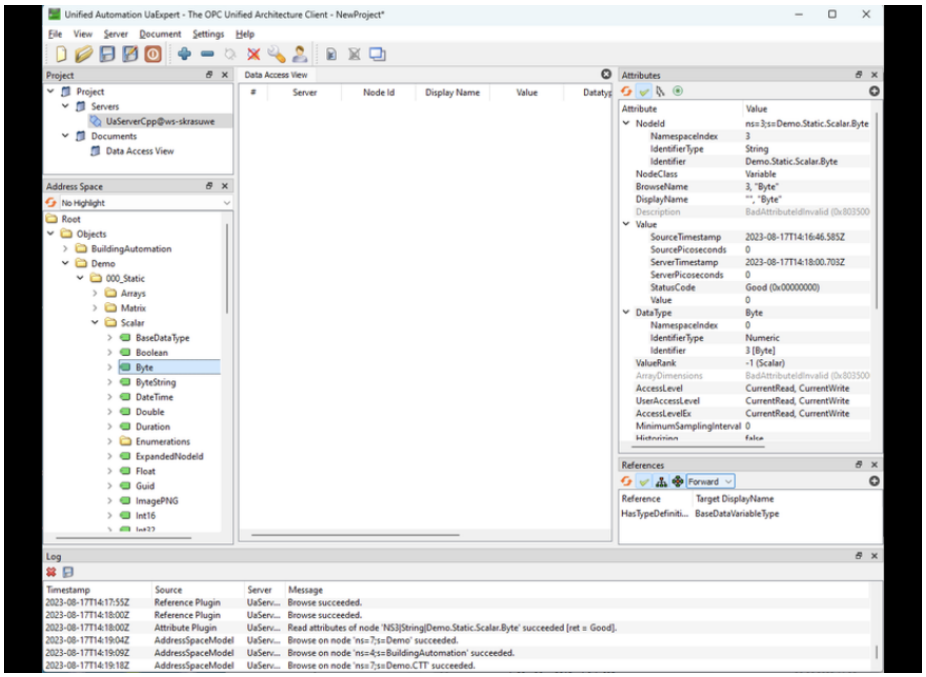


Abbildung 2.1: UaExpert

Listing 2.1: OpcUaClientBehaviour.cs

```
// Copyright (c) Traeger Industry Components GmbH. All Rights Reserved.

using System;
using UnityEngine;
using UnityEngine.UI;

using Opc.UaFx;
using Opc.UaFx.Client;

public class OpcUaClientBehaviour : MonoBehaviour
{
    private OpcClient client;
    private Text statusText;
    private Text statusText4;
    private Text statusText3;
    private OpcSubscription subscription;

    void Start()
    {
        this.statusText = GameObject.Find("statusText").GetComponent<Text
    >();
```

```
    this.statusText4 = GameObject.Find("statusText4").GetComponent<
Text>();
    this.statusText3 = GameObject.Find("statusText3").GetComponent<
Text>();

    this.statusText.text = "Connecting...";
    this.statusText4.text = "Connecting4...";
    this.statusText3.text = "Info3...";

    try
    {
        this.client = new OpcClient("opc.tcp://localhost:4840/");

        this.client.Security.UserIdentity = new OpcClientIdentity("
opcuser1", ".opcuser1");

        this.client.Connect();
        this.statusText.text = "Connected!";

        string[] nodeIds = {
            "ns=6;s=::opctest:mySinValue",
            "ns=6;s=::AsGlobalPV:gSchweibsChange",
            "ns=6;s=::AsGlobalPV:gSchweibsWrite",
            "ns=6;s=::room1:Lampe",
            "ns=6;s=::room1:SwitchValueW",
            "ns=6;s=::room1:SwitchValue"
        };

        this.subscription = this.client.SubscribeNodes();

        for (int index = 0; index < nodeIds.Length; index++)
        {
            var item = new OpcMonitoredItem(nodeIds[index],
OpcAttribute.Value);
            item.DataChangeReceived += HandleDataChanged;
            item.Tag = index;
            item.SamplingInterval = 200;

            this.subscription.AddMonitoredItem(item);
        }

        this.subscription.ApplyChanges();

        this.statusText3.text = "Subscribed!";
    }
    catch (Exception ex)
    {
        if (ex is TypeInitializationException tiex)
            ex = tiex.InnerException;
```

```
            this.statusText.text += Environment.NewLine
                + ex.GetType().Name + ":␣" + ex.Message
                + Environment.NewLine + ex.StackTrace;
        }
    }

    void HandleDataChanged(object sender, OpcDataChangeReceivedEventArgs
    e)
    {
        OpcMonitoredItem item = (OpcMonitoredItem)sender;

        if (item.NodeId.ToString().Contains("ns=6;s=::AsGlobalPV:
    gSchweibsChange"))
        {
            this.statusText.text = e.Item.Value.Value?.ToString() ?? "
    null";
        }
        else if (item.NodeId.ToString().Contains("ns=6;s=::opctest:
    mySinValue"))
        {
            this.statusText4.text = e.Item.Value.Value?.ToString() ?? "
    null";
        }
        else
        {
            Debug.Log("Data␣Change:␣" + item.NodeId + "␣=␣" + e.Item.
    Value);
        }
    }
}
```

Nachdem man das Script geöffnet hat, muss man die Stelle mit der Variable **“this.client = new OpcClient(öpc.tcp://102.168.1.60:4840”)“** finden und die IP-Adresse anpassen, damit eine Verbindung zum Arsim hergestellt werden kann. In Unity soll ein Knopf (Schalter) betätigt werden, welcher dann ein Signal schickt. Wenn der Schalter umgelegt ist, wird im UaExpert ein **“true“** Signal empfangen, ist der Schalter auf seiner Ursprungsposition (Normalzustand), dann empfängt das Programm ein **“false“** Signal.

Kapitel 3

Zusammenfassung

Wir haben in diesem kurzen Sprint/Aufgabe gelernt, wie man die Signale mit dem UaExpert richtig empfängt und verarbeitet. Wie schwierig es ist einen Schalter ohne Dimensionen zu modellieren.